

Dokumentasi (Backend)

Nama : Vincentius Tata Fabian

NPM : 23313015

Kelas : TI23A

1. Saya telah membuat folder api dan lib serta file global.d.ts.

a) Folder api

Untuk folder api digunakan sebagai pengelolaan backend API pada web kami dengan memanfaatkan Next.js App Router. Seluruh endpoint API telah dikelompokkan di dalam folder agar lebih terorganisir dan rapih.

Di dalam folder api, terdapat subfolder yang merepresentasikan fitur utama, yaitu:

- 1) Users – untuk mengelola data pengguna (registrasi, update, hapus, lihat data)
- 2) Auth – untuk mengelola proses autentikasi login
- 3) Transactions – untuk mengelola data transaksi keuangan
- 4) Accounts – untuk mengelola akun keuangan agar dapat mengetahui pengeluaran dan pemasukkan berasal dari mana (m-banking/e-wallet)

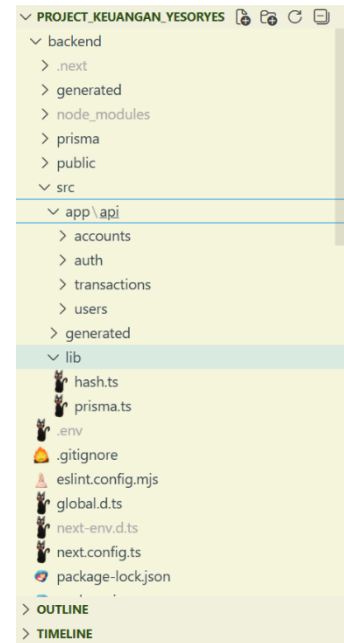
b) Folder lib

Untuk folder lib digunakan sebagai tempat penyimpanan fungsi pendukung. Isi folder lib, yaitu:

- 1) Prisma.ts – digunakan untuk mengatur koneksi ke database menggunakan Prisma, sehingga setiap API dapat mengakses database.
- 2) Hash.ts – digunakan untuk mengamankan password dengan cara mengenkripsi (hash) password saat registrasi dan memverifikasi password saat login.

c) File global.d.ts

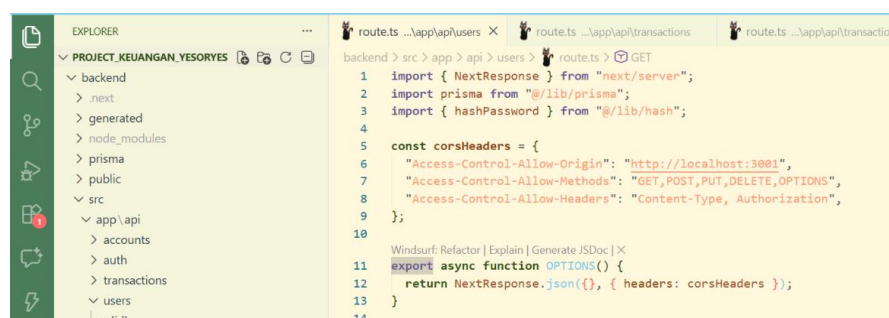
Untuk folder global.d.ts digunakan untuk mendefinisikan tipe data global pada TypeScript.



2. Di dalam folder api saya telah membuat **folder users** yang berisi file hash.ts dan folder [id] yang berisikan file hash.ts juga.



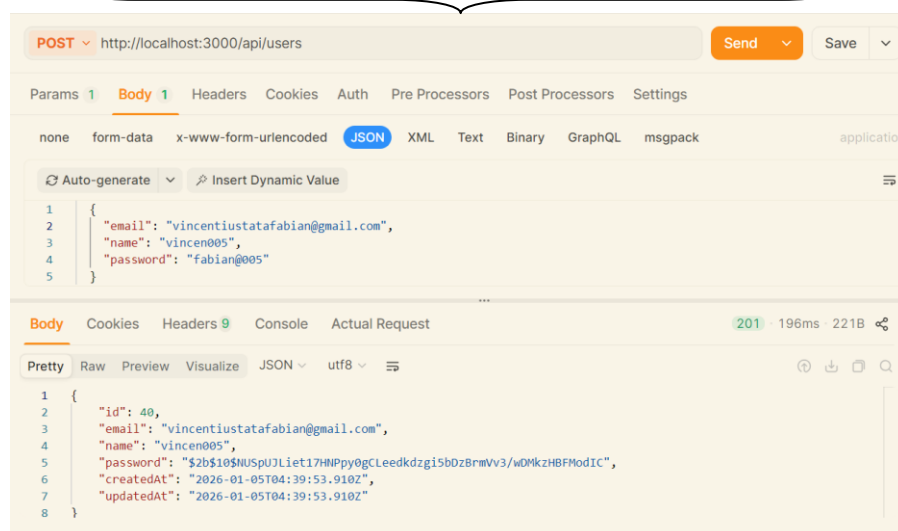
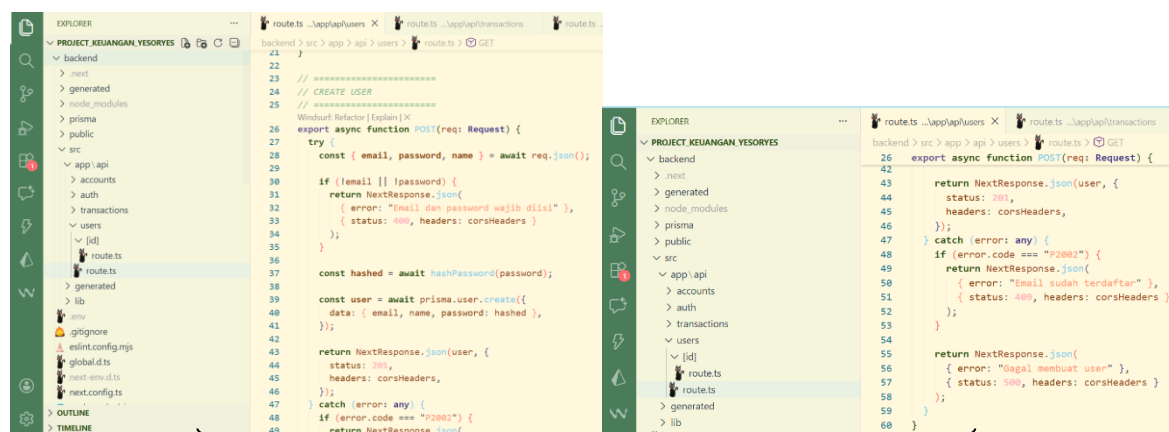
3. (lanjutan dari no 2) Isi file route.ts dari folder users, seperti berikut:



Kode tersebut digunakan untuk menyiapkan respon API Next.js, menghubungkan API dengan database menggunakan Prisma, serta mengamankan password pengguna dengan proses hashing. Pengaturan **CORS** dibuat agar frontend di <http://localhost:3001> diizinkan mengakses API dengan metode tertentu seperti GET, POST, PUT, dan DELETE. Fungsi **OPTIONS** berfungsi menangani preflight request dari browser agar komunikasi frontend dan backend berjalan tanpa error CORS.

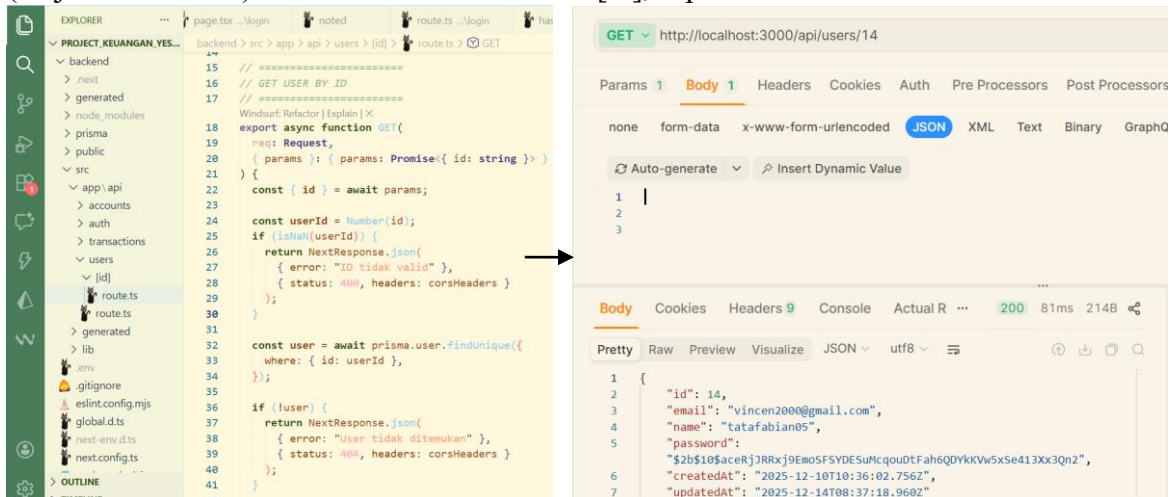


Kode tersebut digunakan untuk mengambil semua data pengguna dari database. Header CORS disertakan agar data tersebut boleh diakses oleh frontend tanpa diblokir browser.



Kode tersebut digunakan untuk membuat user baru ke dalam database melalui metode POST. Password yang dikirim akan di-hash terlebih dahulu agar aman. Jika email sudah terdaftar atau terjadi error, API akan mengirimkan pesan error yang sesuai ke frontend.

4. (lanjutan dari no 2) Isi file route.ts dari folder [id], seperti berikut:



The screenshot displays the source code for the GET route in `route.ts` and the result of a REST client call to `http://localhost:3000/api/users/14`.

Source Code (route.ts):

```

15 // =====
16 // GET USER BY ID
17 // =====
18 export async function GET(
19   req: Request,
20   { params }: { params: Promise<{ id: string }> }
21 ) {
22   const { id } = await params;
23
24   const userId = Number(id);
25   if (!isNaN(userId)) {
26     return NextResponse.json(
27       { error: "ID tidak valid" },
28       { status: 400, headers: corsHeaders }
29     );
30   }
31
32   const user = await prisma.user.findUnique({
33     where: { id: userId },
34   });
35
36   if (!user) {
37     return NextResponse.json(
38       { error: "User tidak ditemukan" },
39       { status: 404, headers: corsHeaders }
40     );
41   }

```

REST Client Response:

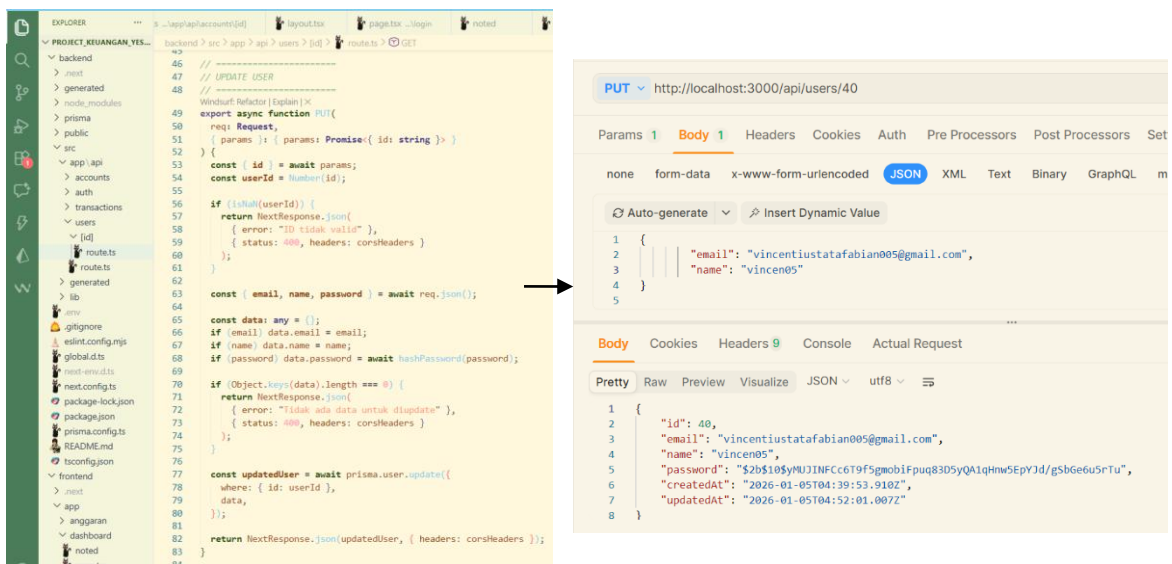
Method: GET
URL: `http://localhost:3000/api/users/14`
Status: 200
Body (JSON):

```

{
  "id": 14,
  "email": "vincen2000@gmail.com",
  "name": "tatafabian05",
  "password": "$2b$10$acerjJRRxj9EmoSFSYDESuMcgouTfAh6QDYkkVw5xSe413Xx3Qn2",
  "createdAt": "2025-12-10T10:36:02.756Z",
  "updatedAt": "2025-12-14T08:37:18.960Z"
}

```

Kode tersebut digunakan untuk mengambil data user berdasarkan ID yang dikirim lewat URL. Kalau ID-nya tidak ada, sistem akan langsung balikin error ke frontend.



The screenshot displays the source code for the PUT route in `route.ts` and the result of a REST client call to `http://localhost:3000/api/users/40`.

Source Code (route.ts):

```

46 // =====
47 // UPDATE USER
48 // =====
49 export async function PUT(
50   req: Request,
51   { params }: { params: Promise<{ id: string }> }
52 ) {
53   const { id } = await params;
54   const userId = Number(id);
55
56   if (!isNaN(userId)) {
57     return NextResponse.json(
58       { error: "ID tidak valid" },
59       { status: 400, headers: corsHeaders }
60     );
61   }
62
63   const { email, name, password } = await req.json();
64
65   const data: any = {};
66   if (email) data.email = email;
67   if (name) data.name = name;
68   if (password) data.password = await hashPassword(password);
69
70   if (Object.keys(data).length === 0) {
71     return NextResponse.json(
72       { error: "Tidak ada data untuk diupdate" },
73       { status: 400, headers: corsHeaders }
74     );
75   }
76
77   const updatedUser = await prisma.user.update({
78     where: { id: userId },
79     data,
80   });
81
82   return NextResponse.json(updatedUser, { headers: corsHeaders });
83
84 }

```

REST Client Response:

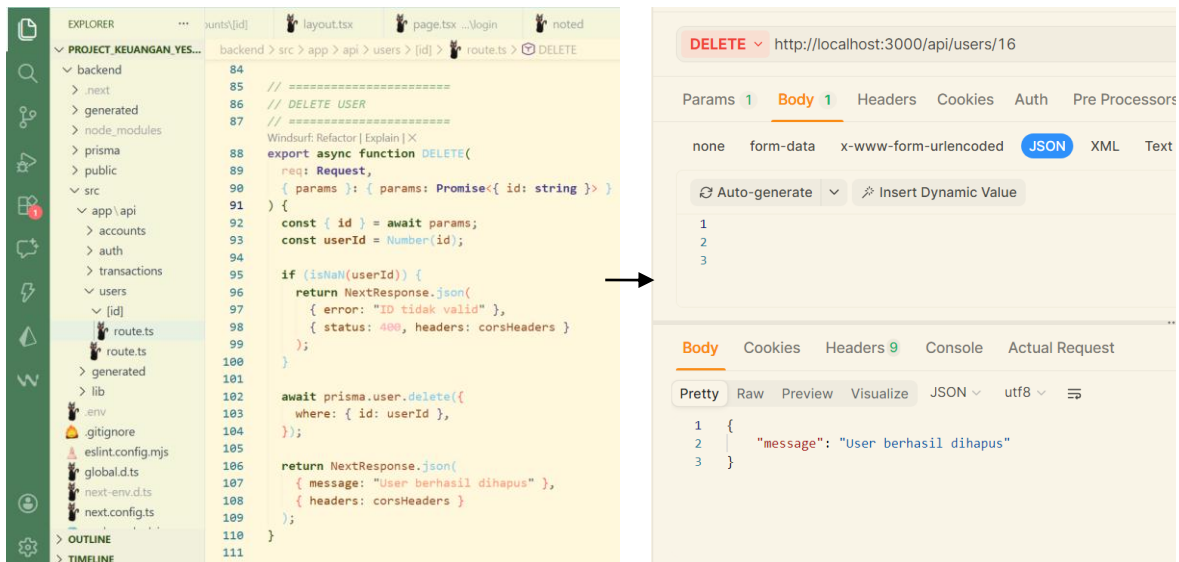
Method: PUT
URL: `http://localhost:3000/api/users/40`
Status: 200
Body (JSON):

```

{
  "id": 40,
  "email": "vincentiustatafabian005@gmail.com",
  "name": "vincen05",
  "password": "$2b$10$yMUJINFc6t9f5gmobiFpuq83D5yQA1qHnw5EpYJd/gSbGe6u5rTu",
  "createdAt": "2026-01-05T04:39:53.910Z",
  "updatedAt": "2026-01-05T04:52:01.007Z"
}

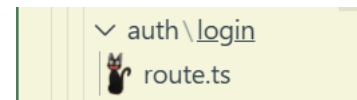
```

Kode tersebut digunakan untuk update data user, dapat berupa email, nama, atau password. Kalau ID salah atau tidak ada data yang diupdate, prosesnya akan dibatalkan dan muncul pesan error.

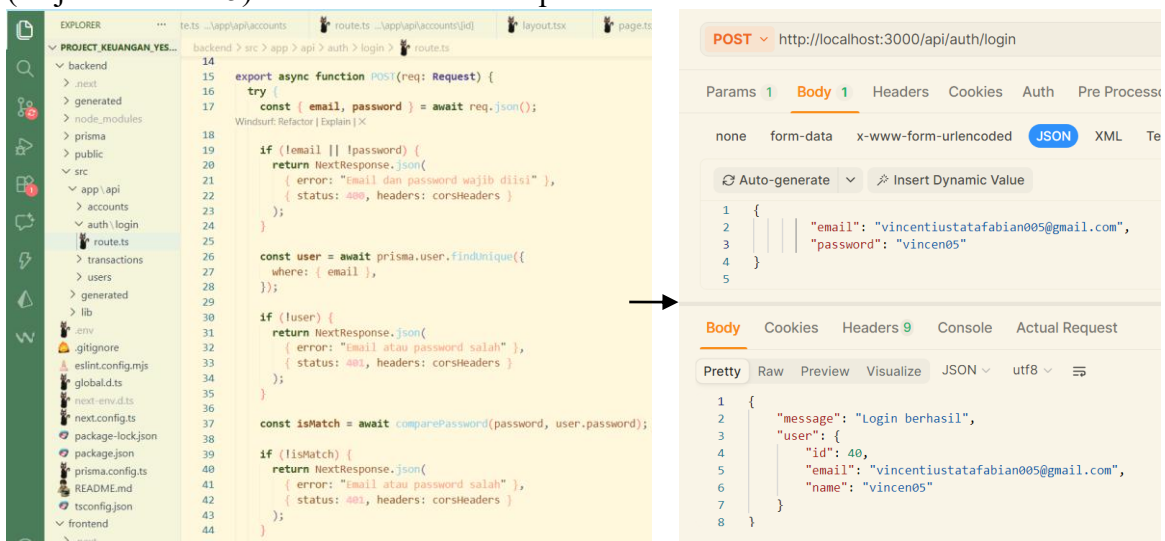


Kode tersebut digunakan untuk menghapus data user yang dikirim melalui URL. Kalau ID-nya tidak valid, proses akan dihentikan, dan jika berhasil maka sistem akan menghapus user dari database lalu mengirim pesan bahwa user berhasil dihapus.

- Selanjutnya saya telah membuat folder `/auth/login` yang berisikan file `route.ts`.

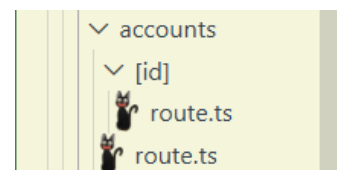


- (lanjutan dari no 5) Isi dari file `route.ts` seperti berikut:

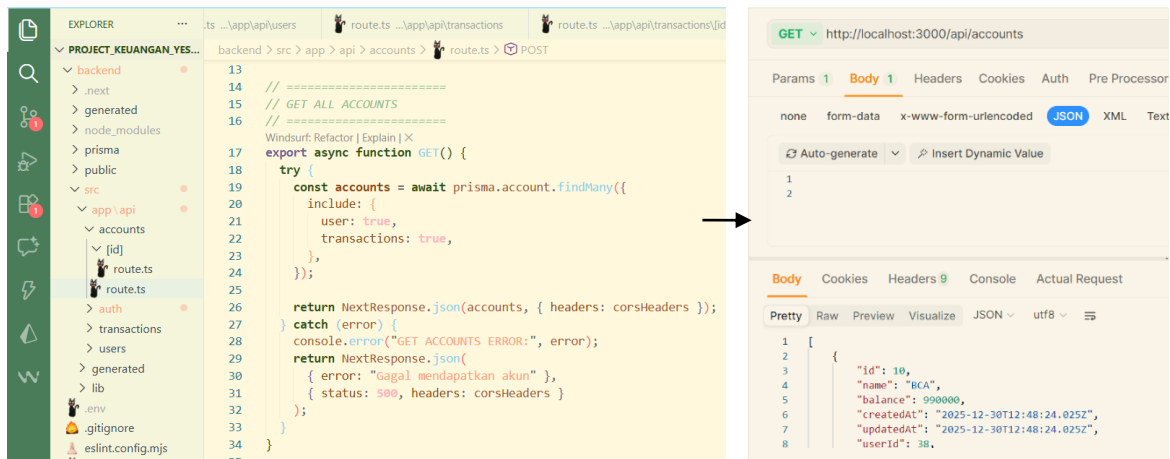


Kode tersebut digunakan untuk proses login user. Sistem akan mengecek apakah email dan password diisi, lalu mencocokkan email dan password ke database, jika cocok maka login dianggap berhasil.

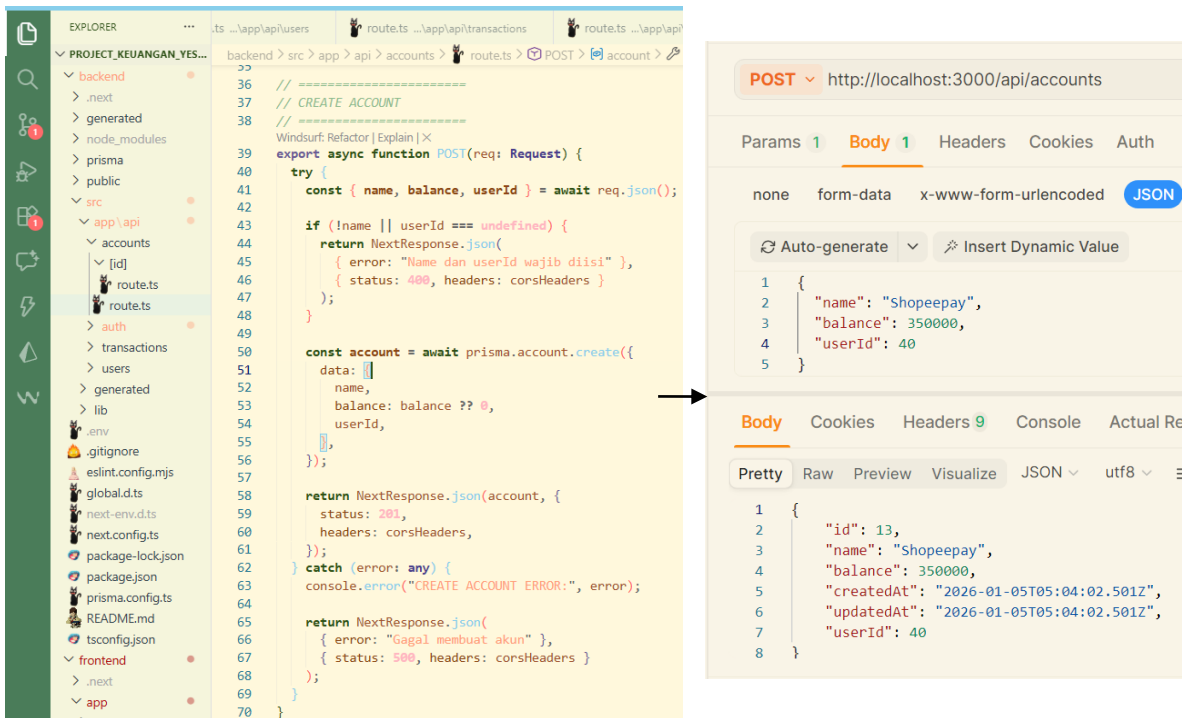
- Selanjutnya saya telah membuat folder `accounts` yang berisikan file `route.ts`.



8. (lanjutan dari no 7) Isi file route.ts dari folder accounts seperti berikut:

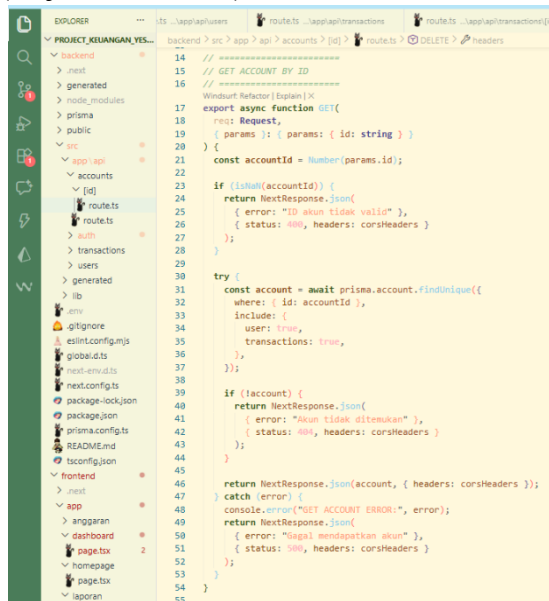


kode tersebut digunakan untuk mengambil semua data akun dari database. Juga mengambil data user pemilik akun dan daftar transaksi tiap akun.



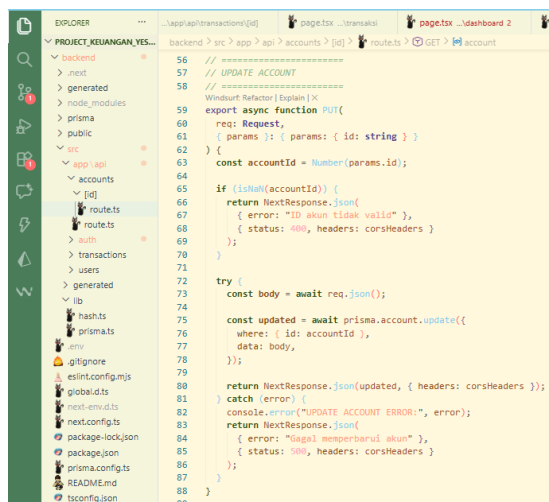
Kode tersebut digunakan untuk membuat akun baru ke database. Sistem menerima name, balance, dan userId, lalu menyimpan akun tersebut sebagai milik user tertentu.

9. (lanjutan dari no 7) Isi file route.ts dari folder [id] seperti berikut:



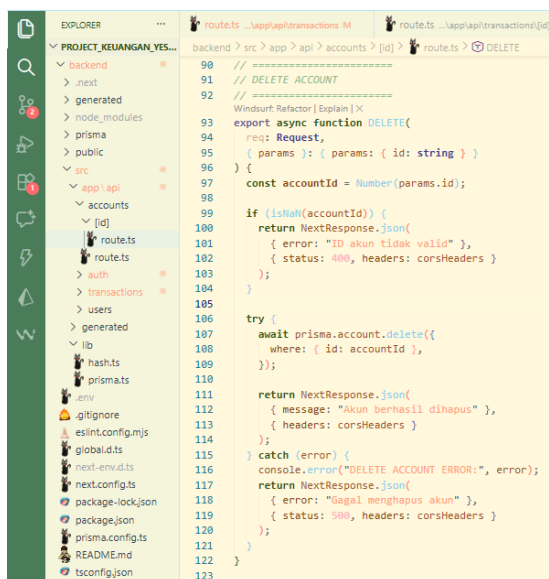
```
14 // =====
15 // GET ACCOUNT BY ID
16 // =====
17 export async function GET(
18   req: Request,
19   { params }: { params: { id: string } }
20 ) {
21   const accountId = Number(params.id);
22
23   if (!isNaN(accountId)) {
24     return NextResponse.json(
25       { error: "ID akun tidak valid" },
26       { status: 400, headers: corsHeaders }
27     );
28   }
29
30   try {
31     const account = await prisma.account.findUnique({
32       where: { id: accountId },
33       include: {
34         user: true,
35         transactions: true,
36       },
37     });
38
39     if (!account) {
40       return NextResponse.json(
41         { error: "Akun tidak ditemukan" },
42         { status: 404, headers: corsHeaders }
43       );
44     }
45
46     return NextResponse.json(account, { headers: corsHeaders });
47   } catch (error) {
48     console.error("GET ACCOUNT ERROR:", error);
49     return NextResponse.json(
50       { error: "Gagal mendapatkan akun" },
51       { status: 500, headers: corsHeaders }
52     );
53   }
54 }
55
```

Kode tersebut digunakan untuk mengambil data akun berdasarkan ID. Juga mengembalikan data user pemilik akun dan semua transaksi yang terkait sama akun tersebut.



```
56 // =====
57 // UPDATE ACCOUNT
58 // =====
59 export async function PUT(
60   req: Request,
61   { params }: { params: { id: string } }
62 ) {
63   const accountId = Number(params.id);
64
65   if (!isNaN(accountId)) {
66     return NextResponse.json(
67       { error: "ID akun tidak valid" },
68       { status: 400, headers: corsHeaders }
69     );
70   }
71
72   try {
73     const body = await req.json();
74
75     const updated = await prisma.account.update({
76       where: { id: accountId },
77       data: body,
78     });
79
80     return NextResponse.json(updated, { headers: corsHeaders });
81   } catch (error) {
82     console.error("UPDATE ACCOUNT ERROR:", error);
83     return NextResponse.json(
84       { error: "Gagal memperbarui akun" },
85       { status: 500, headers: corsHeaders }
86     );
87   }
88 }
89
```

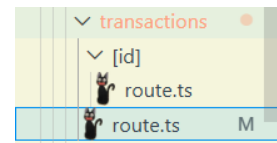
Kode tersebut digunakan untuk update data akun. Data yang dikirim dari request langsung dipakai untuk mengubah isi akun di database.



```
90 // =====
91 // DELETE ACCOUNT
92 // =====
93 export async function DELETE(
94   req: Request,
95   { params }: { params: { id: string } }
96 ) {
97   const accountId = Number(params.id);
98
99   if (!isNaN(accountId)) {
100     return NextResponse.json(
101       { error: "ID akun tidak valid" },
102       { status: 400, headers: corsHeaders }
103     );
104   }
105
106   try {
107     await prisma.account.delete({
108       where: { id: accountId },
109     });
110
111     return NextResponse.json(
112       { message: "Akun berhasil dihapus" },
113       { headers: corsHeaders }
114     );
115   } catch (error) {
116     console.error("DELETE ACCOUNT ERROR:", error);
117     return NextResponse.json(
118       { error: "Gagal menghapus akun" },
119       { status: 500, headers: corsHeaders }
120     );
121   }
122 }
123
```

Kode tersebut digunakan untuk menghapus akun. Sistem akan mengecek dulu ID-nya valid atau tidak, kalau valid akun di database dihapus dan balikin pesan “berhasil”, tapi kalau gagal bakal balikin error.

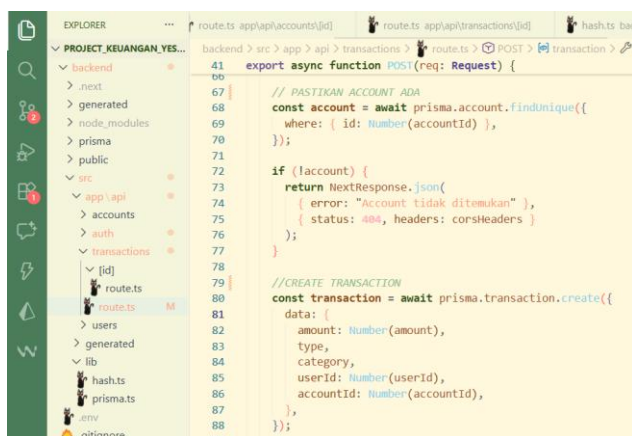
10. Selanjutnya saya telah membuat folder transactions yang berisikan file route.ts.



11. (lanjutan dari no 10) Isi file route.ts dari folder transactions seperti berikut:



Kode tersebut digunakan untuk mengambil semua data transaksi dari database. Transaksi diurutkan dari yang paling baru, sekaligus mengambil data akun terkait supaya dashboard tau transaksi itu milik akun yang mana.



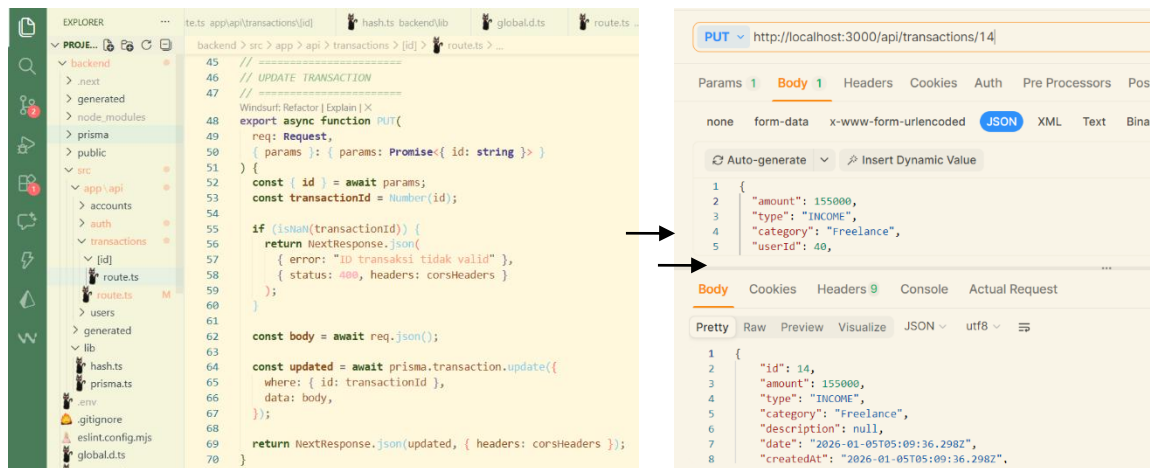
Kode untuk cek akun digunakan untuk mengecek terlebih dahulu apakah akun dengan *accountId* yang dikirim benar-benar ada di database. Kalau akunnya tidak ketemu, proses langsung dihentikan.

Kalau akunnya ada, kode ini akan membuat data transaksi baru ke database. Data yang disimpan berupa jumlah uang, tipe transaksi, kategori, user, dan akun yang dipakai.

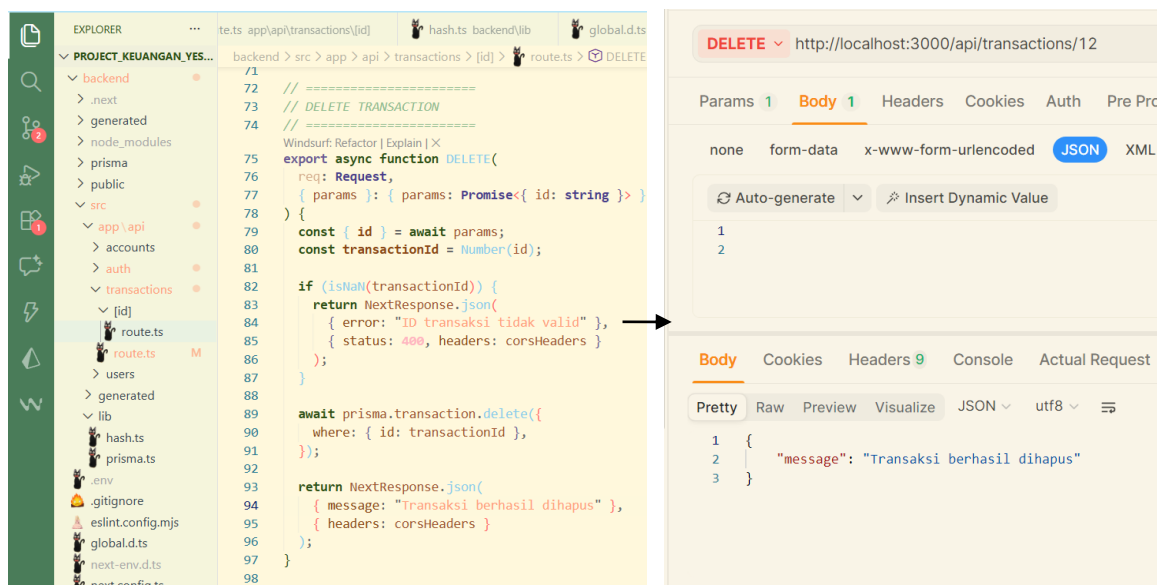


Kode ini digunakan untuk mengupdate saldo akun setelah transaksi dibuat, di mana saldo akan bertambah kalau tipe transaksi **INCOME** dan berkurang kalau **EXPENSE**. Setelah itu API mengembalikan data transaksi yang berhasil dibuat, dan kalau terjadi error.

12. (lanjutan dari no 10) Isi file route.ts dari folder [id] seperti berikut:



Kode tersebut digunakan untuk mengubah data transaksi berdasarkan ID transaksi yang dikirim lewat URL. API akan mengambil data baru dari request body, lalu mengupdate transaksi tersebut di database dan mengembalikan hasil update-nya.



Kode tersebut digunakan untuk menghapus data transaksi berdasarkan ID. ID akan dicek dahulu valid atau tidak, kalau valid maka transaksi di database dihapus dan server memberi respon kalau transaksi berhasil dihapus.

13. Pada folder lib saya telah membuat dua file yaitu `prisma.ts` dan `hash.ts`. Dengan detail file seperti berikut:
file `prisma.ts`:


```

1 import { PrismaClient } from "@prisma/client";
2
3 declare global {
4   // Biarkan TS tahu bahwa global.prisma itu ada
5   var prisma: PrismaClient | undefined;
6 }
7
8 export const prisma =
9   global.prisma ||
10  new PrismaClient({
11    log: ["query"],
12  });
13
14 // Simpan prisma ke global pada environment development
15 if (process.env.NODE_ENV !== "production") global.prisma = prisma;
16
17 export default prisma;
18

```

File `prisma.ts` digunakan sebagai penghubung utama web ke database lewat Prisma.

```

1 import bcrypt from "bcryptjs";
2
3 /**
4  * Hash password menggunakan bcrypt
5  * @param password password plain
6  * @returns string hashed password
7  */
8 export async function hashPassword(password: string): Promise<string> {
9   const saltRounds = 10;
10  return await bcrypt.hash(password, saltRounds);
11 }
12
13 /**
14  * Verifikasi password
15  * @param password password plain
16  * @param hashed hashed password dari database
17  * @returns boolean
18  */
19 export async function verifyPassword(password: string, hashed: string): Promise<boolean> {
20  return await bcrypt.compare(password, hashed);
21 }
22
23 /**
24  * Generate JWT token
25  * @param payload data yang akan disimpan di token
26  * @returns string JWT token
27  */
28 export async function generateToken(payload: any): Promise<string> {
29  return await jwt.sign(payload, process.env.JWT_SECRET as string, { expiresIn: '1h' });
30 }
31
32 /**
33  * Decode JWT token
34  * @param token JWT token
35  * @returns any data yang disimpan di token
36  */
37 export async function decodeToken(token: string): Promise<any> {
38  return await jwt.verify(token, process.env.JWT_SECRET as string);
39 }
40

```

File `hash.ts` digunakan untuk mengamankan password dengan cara mengubah password asli menjadi bentuk hash sebelum disimpan ke database. Selain itu, file ini juga berfungsi mengecek kecocokan password saat login.

14. Selanjutnya saya menambahkan fungsi API pada frontend.

a) Pada form **registrasi** seperti berikut:

```

30
31 const res = await fetch("http://localhost:3000/api/users", {
32   method: "POST",
33   headers: { "Content-Type": "application/json" },
34   body: JSON.stringify(form),
35 });
36

```

API pada halaman registrasi berfungsi untuk mengirim data nama, email, dan password yang diinput pengguna dari

frontend ke backend melalui endpoint `/api/users` menggunakan metode POST agar data tersebut dapat diproses dan disimpan sebagai akun baru.

b) Pada form **login** seperti berikut:

```

65 const res = await fetch("http://localhost:3000/api/auth/login", {
66   method: "POST",
67   headers: {
68     "Content-Type": "application/json",
69   },
70   body: JSON.stringify({
71     email: form.email,
72     password: form.password,
73   }),
74 });
75

```

API tersebut berfungsi untuk mengirim data email dan password dari form login ke backend melalui endpoint `/api/auth/login` menggunakan metode POST

untuk proses autentikasi pengguna.